Digitisation - Environment Propagation Mapping

Status	PEER REVIEW
Impact	HIGH
Driver	@manu.srinivasa@greenflag.com
Approver	@john.mcdonnell@greenflag.com @franco.parente@greenflag.com
Decision Contributors	@franco.parente@greenflag.com @james.wright1@greenflag.com
Informed	@james.wright1@greenflag.com @franco.parente@greenflag.com
Due date	21 Aug 2025

Executive Summary

The GF Customer Web project is a strategic initiative designed to deliver a robust, scalable, and secure digital platform for Green Flag's customers. The initiative aligns with DLG's engineering standards and emphasizes automation, environment consistency, and advanced deployment strategies. By leveraging a modern CI/CD pipeline, infrastructure as code (IaC), and automation-first principles, the project ensures consistency, traceability, and compliance across all environments.

Introduction

The project's primary objective is to establish seamless propagation of code, configuration, and infrastructure changes across all environments—Development (DEV), Testing, Staging, and Production (PROD-OFFLINE/ONLINE). This is achieved by adopting best practices from the Nova Project, focusing on automation, environment parity, and secure deployment. The approach leverages tools such as **GitHub Actions** for CI/CD, **Terraform** for IaC, **AWS ECS Fargate** for hosting Next.js Frontend and **AWS Lambda with API Gateway** for Fastify BFF with auto-scaling capabilities.

Background and Context

The GF Customer Web project integrates modern CI/CD methodologies, infrastructure standards, and operational strategies. The project's environment propagation mapping ensures structured movement of code and infrastructure changes, maintaining consistency and compliance with DLG's engineering standards.

Foundational Principles

- **CI/CD Pipelines in GitHub Actions**: All build and deployment workflows are managed in GitHub Actions, with self-hosted runners for long-running tasks. For more details: Digitisation - Environment Propagation Mapping | Key Contacts and References
- **Infrastructure as Code (IaC)**: Terraform is used for provisioning and managing infrastructure, ensuring environment parity and simplifying management. For more details: Digitisation - Environment Propagation Mapping | Key Contacts and References
- **Access Control and Deployment Permissions**: Deployment is managed via GitHub with codeowner approvals. For more details: Access Control & Deployment Permissions with GitHub and CODEOWNERS
- **Change Management Automation**: Integration with ServiceNow automates change request creation and checks. For more details: Change Management Automation with ServiceNow
- **Feature Flags and Rollback Strategies**: Feature flags are planned for future iterations, with blue/green deployments as the primary rollback mechanism. Any excepts follow through fix forward approach in the PROD-ONLINE environment. For more details: Feature Flags and Rollback strategies with blue/green deployments as the primary rollback mechanism
- **Documentation and Knowledge Sharing**: All technical decisions and diagrams are documented in Confluence for transparency and accessibility.
- **Disaster recovery (DR)** across multiple AWS regions is out of scope.

Methodology

The project methodology is rooted in automation, consistency, and traceability, ensuring all environments are aligned with organizational standards.

1. CI/CD Pipeline Design

- **GitHub Actions**: All CI/CD workflows are managed in GitHub Actions, eliminating legacy tools. Digitisation - Environment Propagation Mapping | Propagation Workflow
- **GitHub Hosted and Self-Hosted Runners**: Provisioned for long-running workflows. Digitisation - Environment Propagation Mapping | Propagation Workflow

- **Branching Strategy**: Trunk Based Structured model with `main` for production, `feature` for new work(development). Branching Strategy - Trunk based Structured Model

2. Infrastructure as Code (IaC)

- **Terraform**: Used for all infrastructure provisioning, with modules and workspaces for environment-specific configurations (Terraform Standards).
- **Automated Provisioning**: Infrastructure changes are controlled and deployed through the CI/CD pipeline.

3. Deployment and Environment Management

- **Environment Mapping**: Changes propagate through DEV → TEST → STAGING → PROD(OFFLINE) → PROD(ONLINE), with automated tests and approvals at each stage. Digitisation - Environment Propagation Mapping | Propagation Workflow
- **Deployment Permissions**: Managed via GitHub CODEOWNERS, with approval processes modelled after Motor Inc. Access Control & Deployment Permissions with GitHub and CODEOWNERS
- **Blue/Green Deployments**: Used in production for zero-downtime releases and rapid rollback. Digitisation - Environment Propagation Mapping | Production (PROD OFFLINE), Digitisation - Environment Propagation Mapping | Production (PROD ONLINE)

4. Testing and Quality Assurance

- **Automated Testing**: Unit, integration, and E2E tests are triggered on environment promotion (Digitisation - UI and BFF Testing Strategy).
- **Security Scanning**: Automated security scans are integrated into the pipeline(Codacy for SAST(Mycro Service), SonarQube for SAST(UI+BFF), Rapid7 for DAST, and Qualys for PCI scanning).

5. Change Management and Documentation

- **Automated/Manual SNOW Change Requests**: ServiceNow integration automates change requests. Change Management Automation with ServiceNow (SNOW Integration).
- **Documentation**: All technical decisions and outcomes are documented in Confluence (KDD Documentation Standards).

6. Feature Flags and Future Considerations

- **Feature Flags**: Feature Flags are a planned enhancement (referencing MCH's LaunchDarkly pattern) to enable or disable features at runtime without redeploying code. Feature Flags and Rollback strategies with blue/green deployments as the primary rollback mechanism

7. Application Deployment

- **Frontend**: Deployed to ECS Fargate using golden image patterns.
- **BFF**: Deployed as AWS Lambda with provisioned concurrency and AWS X-Ray for tracing (AWS Lambda Provisioned Concurrency, AWS X-Ray Documentation).

8. Automation and Observability

- **Automate Everything**: All repeatable processes are automated.
- **Monitoring and Alerts**: AWS Cloud Watch and X-Ray provide operational visibility along with AppDynamics for App Monitoring

---

Environment Propagation Mapping

The project operates across four primary environments, each with specific configurations and controls to ensure quality and minimize risk.

Environment Overview and Configurations

Environment	Purpose	Infra Provisioning	Deployment Trigger	Runner Type	Approval Required	AWS Account	Notes
DEV	Developer	Terraform	PR/Merge to main	GitHub-hosted	No	AWS_DEV	Automated Unit/Integration/E2E Testing, Static analysis, terraform plan
TEST	QA	Terraform	PR/Merge to	Self-hoste	No	AWS_QA	QA and

			main	d			Perf/E2E Tests, Security scans
STAGING	Pre-Prod	Terraform	Manual trigger	Self-hosted	Yes (Lead)	AWS_STAGING	UAT, exploratory/Perf tests, Security scans
PROD(OFFLINE)	Prod	Terraform	Manual trigger	Self-hosted	Yes (PO/Lead)	AWS_PROD	Smoke Test, Blue/Green, SNOW integration, Final Validation, not customer facing
Production(ONLINE)	Live	Terraform	Manual trigger	Self-hosted	Yes (PO/Lead)	AWS_PROD	Blue/Green switch, SNOW close, Live, Monit

							oring, Customer facing

Propagation Workflow

The propagation of code and infrastructure changes follows this path:

DEV → TEST → STAGING → PROD (OFFLINE) → PROD (ONLINE)

Each stage is designed to increase confidence, reduce risk, and ensure compliance with DLG and Green Flag standards.

Development (DEV)

Purpose:

- Active development, initial validation, and fast feedback.

How it works:

1. **Code Commit & PR:**
   - Developers push code to feature branches and open Pull Requests (PRs) against `main`.
2. **GitHub Actions Trigger:**
   - On PR or merge to `main`, GitHub Actions (using GitHub-hosted runners) automatically run:
     - Linting
     - Unit tests (with mocked upstream services)
     - Integration tests (with upstream SI microservice)
     - Static analysis (e.g., Codacy(Mycro Service) and SonarQube(UI+BFF))
3. **Terraform Plan:**
   - If infrastructure changes are detected, a `terraform plan` is run (but not applied).
4. **Feedback:**
   - Results are posted to the PR for developer review.

Access Control:

- Restricted to developers

Automation:

- All steps are automated via GitHub Actions.
- No manual approvals required at this stage.

Test (QA)

Purpose:

- QA and performance testing in a controlled environment.

How it works:

1. **Manual Promotion:**
   - After successful DEV validation, a manual trigger (via workflow_dispatch or similar) promotes the build to TEST.
2. **GitHub Actions Trigger:**
   - Self-hosted runners are used for longer-running jobs.
   - Actions run:
     - Infrastructure provisioning (`terraform apply`)
     - Application deployment (ECS Fargate for frontend, Lambda for BFF)
     - QA and performance tests (Integration with upstream SI microservice, E2E, load)
     - Security scans (Rapid7, Qualys)
3. **Feedback:**
   - Test results are reviewed by QA.

Access Control:

- Strict, with MFA.

Automation:

- Infrastructure and deployment are automated.
- Manual trigger required to start the TEST workflow.

Staging (Pre-Prod)

Purpose:

- Pre-production validation (UAT, Exploratory testing against staging of real upstream services. Integration tests in this environment can use staging of upstream services, performance).

How it works:

1. **Manual Promotion:**
   - After QA sign-off, a manual trigger promotes the build to STAGING.
2. **GitHub Actions Trigger:**
   - Self-hosted runners execute:
     - Infrastructure provisioning (mirrors PROD)

    - Application deployment(ECS Fargate for frontend, Lambda for BFF, Integration with staging microservice)
    - UAT, exploratory, and performance tests
    - Security scans
3. **Approval:**
    - PO or Lead Engineer must approve before deployment.
4. **Feedback:**
    - UAT and test results are reviewed.

Access Control:

- Strict, with MFA.

Automation:

- All technical steps are automated.
- Manual approval required before deployment.

Production (PROD-OFFLINE)

Purpose:

- Final validation in a production-like environment, not customer-facing.

How it works:

1. **Manual Promotion:**
    - After STAGING sign-off, a manual trigger promotes to PROD-OFFLINE.
2. **GitHub Actions Trigger:**
    - Self-hosted runners execute:
        - Infrastructure provisioning (Terraform)
        - Blue/green deployment (deploy new version alongside current)
        - Smoke tests and final validation
        - ServiceNow integration for change management (automated change request creation/checks)
3. **Approval:**
    - Multi-level approval (PO, QA Lead, Release Manager).
4. **Feedback:**
    - Results are reviewed; if issues, rollback is possible.

Access Control:

- Strict, with MFA.

Automation:

- All deployment and validation steps are automated.
- Manual approval and promotion required.

<u>Production (PROD-ONLINE)</u>

Purpose:

- Live, customer-facing environment.

How it works:

1. **Manual Promotion:**
   - After warranty period and final validation in PROD-OFFLINE, a manual trigger promotes to PROD-ONLINE.
2. **GitHub Actions Trigger:**
   - Self-hosted runners execute:
     - Blue/green deployment (switch traffic to new version. Green becomes live, Blue is kept as backup)
     - Final smoke tests
     - ServiceNow integration (close change request)
     - Monitoring setup (AWS CloudWatch, X-Ray)
3. **Approval:**
   - Multi-level approval required.
4. **Feedback:**
   - Monitoring for issues; blue/green enables immediate rollback if needed. Fix forward if we have any rollback issues.

Access Control:

- Strict, with MFA.

Automation:

- All technical steps are automated.
- Manual approval and promotion required.

CI/CD Implementation

Build and Deployment Workflows

- **GitHub Actions**: All workflows managed in GitHub Actions, with self-hosted runners for long-running tasks.
- **Pipeline Standards**: Adheres to(DLG or Green Flag's pipeline standards.)
- **Automated Testing**: Unit, integration, and E2E tests at each stage.

Infrastructure as Code (IaC)

- **Terraform**: Used for all environments, with modules and workspaces for environment-specific configs.
- **Automated Provisioning**: Infrastructure changes are automated and -controlled.

Deployment Permissions and Approvals

- **Codeowners**: Deployment managed via GitHub, with codeowner approvals (Motor Inc. Deployment Process).
- **Branching Strategy**: `main` for production, `dev` for development, feature branches for new work.

Change Management

- **ServiceNow Integration**: Automated change request creation/checks (SNOW Integration).

Feature Flags

- **Future Integration**: Plans to use LaunchDarkly, referencing MCH's implementation (Feature Flags in MCH).

Rollback and Release Strategy

- **Blue/Green Deployments**: Enables rapid rollback during warranty period (Blue/Green Deployment Process).
- **Monitoring**: AWS CloudWatch and X-Ray for post-deployment monitoring (AWS X-Ray Documentation).

Infrastructure as Code

Terraform Implementation

- **Environment-Specific Configuration**: Modules and workspaces maintain parity and allow customization.
- **Automated Provisioning**: Integrated with CI/CD for error-free deployments.

- **Access Control**: Managed via GitHub, with codeowner approvals.
- **Change Management**: Automated SNOW integration for change requests.

Application Deployment

- **Frontend**: ECS Fargate with golden image patterns (Example Dockerfile).
- **BFF**: AWS Lambda with provisioned concurrency and X-Ray tracing (AWS Lambda Provisioned Concurrency, AWS X-Ray Documentation).

Automation and Documentation

- **Automation**: All processes, including infrastructure provisioning, are automated.
- **Documentation**: Maintained in Confluence, covering KDDs, diagrams, and outcomes (KDD Documentation Standards).

Automation and Documentation

Automation

- **CI/CD Pipelines**: Managed in GitHub Actions, with self-hosted runners for extended tasks.
- **IaC with Terraform**: Ensures environment consistency and automated provisioning.
- **Automated Testing and Deployment**: Integrated at every pipeline stage.
- **Change Management**: Automated via ServiceNow.
- **Feature Flags**: Planned for future scalability.
- **Blue/Green Deployment**: Enables zero-downtime releases and safe rollbacks.
- **Comprehensive Automation**: All repeatable processes are automated.

Documentation

- **Centralized Repository**: All technical decisions, KDDs, and diagrams are documented in Confluence.
- **Environment Mapping Table**: Clearly outlines environment purposes, triggers, and approvals.
- **Continuous Updates and Peer Reviews**: Documentation is regularly updated and peer-reviewed.
- **Stakeholder Visibility**: Structured for clarity and accessibility.

Challenges and Solutions

Challenge	Solution

CI/CD Pipeline Complexity	Fully implemented in GitHub Actions, with self-hosted runners for long-running workflows and automated testing at every stage.
Infrastructure Consistency	Terraform for IaC, with modules and workspaces for environment-specific configs, automated provisioning via CI/CD.
Deployment Permissions	Managed via GitHub Codeowners, with iterative improvements to the approval process and enforced branch protection.
Change Management Automation	Automated SNOW integration, modeled after MCH Motor Claims Hub, with stakeholder collaboration.
Feature Flag Implementation	Future planning referencing MCH's pattern and LaunchDarkly, with detailed documentation for readiness.
Rollback and Release Strategy	Blue/green deployment in production, prioritizing fix-forward strategy and rapid rollback. Disaster recovery (DR) across multiple AWS regions is out of scope and will be addressed in later phases of the project.
Application Deployment Optimization	Frontend on ECS Fargate with golden images, BFF as AWS Lambda with provisioned concurrency and X-Ray, performance testing with tools like Locust.

Documentation and Knowledge Sharing	Comprehensive Confluence documentation, peer reviews, and centralized repository for all project artifacts.

Conclusion

The **Environment Propagation Mapping for GF Customer Web** project exemplifies a modern, scalable, and efficient approach to software delivery. By leveraging proven patterns from the Nova partnership and adopting cutting-edge CI/CD and infrastructure practices, the project ensures:

- **Operational Efficiency**: Automation and standardized workflows reduce manual effort and accelerate deployments.
- **Scalability**: Terraform and ECS Fargate enable seamless scaling.
- **Resilience**: Blue/green deployments and robust rollback strategies minimize downtime.
- **Quality Assurance**: Integrated testing and automated quality gates ensure high-quality releases.
- **Future-Proofing**: Planned feature flag integration and advanced CI/CD practices support long-term adaptability.

Continuous Improvements:

1. **Expand Automation**: Continue automating manual processes, especially in change management and testing.
2. **Enhance Observability**: Leverage AWS X-Ray and monitoring tools for deeper insights.
3. **Iterate Deployment Processes**: Refine approval processes based on lessons learned.
4. **Adopt Feature Flags**: Integrate feature flags for safer, flexible releases.
5. **Strengthen Documentation**: Maintain up-to-date, accessible documentation for all technical decisions and workflows.

Key Contacts and References

- **Deployment Process (Motor Inc.)**: (Motor Inc. Deployment Process). Niall Crouch, Alex Vest
- **SNOW Integration (MCH)**: Steve Briscoe, Daniel Hodgkins
- **Feature Flags (MCH)**: Feature Flags Implementation
- **Terraform Standards**: Terraform General Standards

- **GitHub Actions Pipelines**: GitHub Pipelines, Self-Hosted Runner Guidance
- **KDD Documentation Standards**: KDD003 - Smart Symptoms DB Technology
- **Self Hosted Runner:** Self-Hosted Runner Guidance
- **Blue/Green Deployment:** https://greenflag.atlassian.net/wiki/x/lIBIr Request access
- **Feature Flags:** MCH's implementation Feature Flags Implementation
- **Test Strategy:** Test Strategy - DIGITISATION

Environment Mapping Table:

Refer to Digitisation - UI → BFF → Upstream API Environment Mapping